Pocket Calculator

Table of Contents:

1. CHAPTER 1 : PROJECT SPECIFICATIONS **2**
2. CHAPTER 2 : DESIGN **2**
2.1. Black Box 2
2.2. Control and Execution Unit 3
2.3. Resources 4
2.4. State diagram 6
3. CHAPTER 3 : USER MANUAL **8**
4. CHAPTER 4 : DESIGN SOLUTIONS JUSTIFICATION **8**
5. CHAPTER 5 : FURTHER DEVELOPMENT **8**
6. CHAPTER 6 : BIBLIOGRAPHY **8**

Student: Hideg Mark-Alex

CHAPTER 1 : PROJECT SPECIFICATIONS

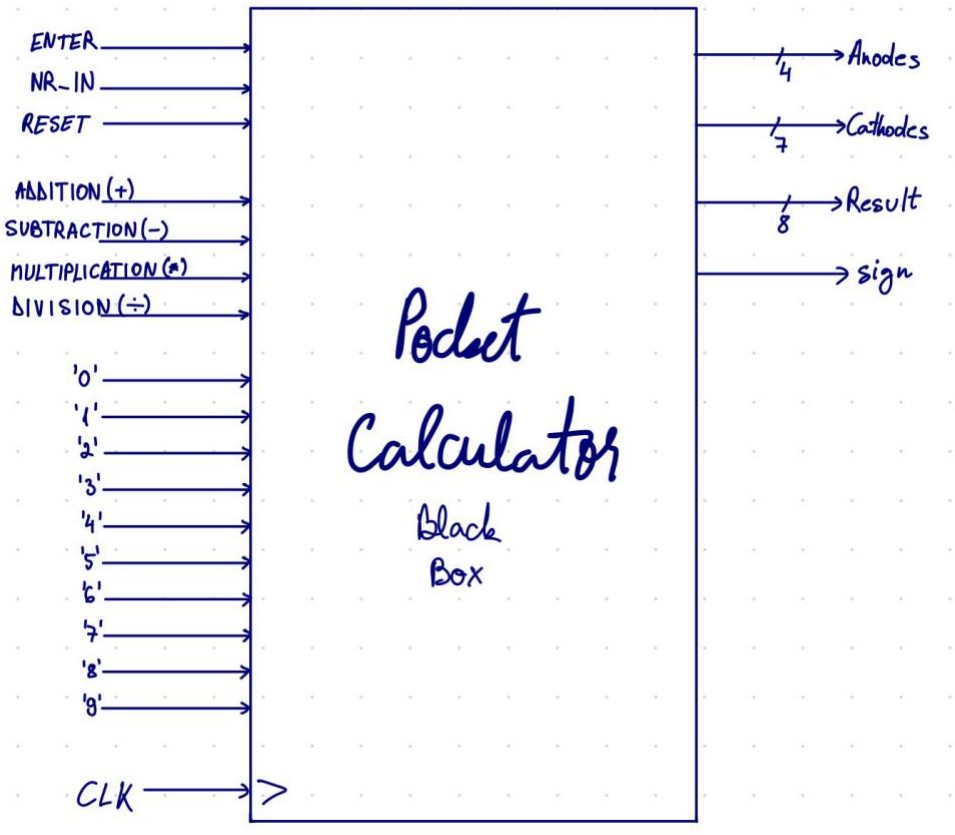
Design a pocket computer with basic arithmetic operations (addition, subtraction, multiplication, division). Multiplication and division operations will be implemented using specific algorithms, not language operators. The operands are represented by 8 bits with a sign. Operants and operators will be entered sequentially in decimal form. The 7-segment displays on the FPGA boards will be used. The project will be carried out by 1 student.

Operating Procedure:

- **Initial State**: The calculator starts in standby, displaying zero, with all operands reset.
- **Number Entry**: Users toggle slide switches for digits 0–9, activating NR_IN to confirm each digit, building numbers sequentially.
- **Operator Selection**: Users activate plus, minus, ori, div to select an operation.
- **Computation**: Toggling Enter triggers the calculation, displaying the result on the 7-segment display.
- **Reset**: Pressing Reset resets the calculator to standby.

CHAPTER 2 : DESIGN

2.1 Black Box

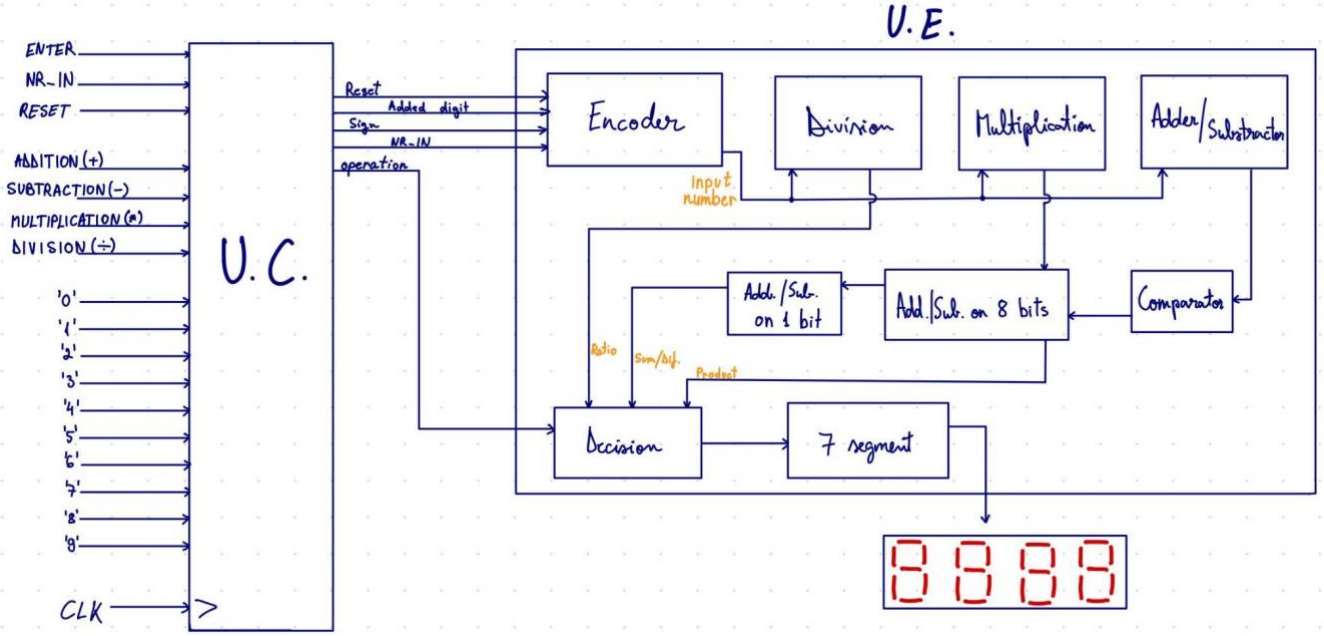


The inputs are:

- 0,1,2,3,4,5,6,7,8 : representing the digits which the user will introduce;
- RESET : will initialize the numbers with 0; - +,-,*,/ : the operands
- ENTER : displays the result
- NR_IN : confirms an introduced digit and the follow-up of another operand.
- For the implementation on the FPGA board another input will be needed: the clock signal (CLK)

The outputs are represented by 4 anodes and 7 cathodes that will blink one by one at high frequency so that the human eye can observe the difference. Moreover, we will introduce 8 outputs for the 8-bit number that is calculated, 4 outputs for the remainder and one for the sign

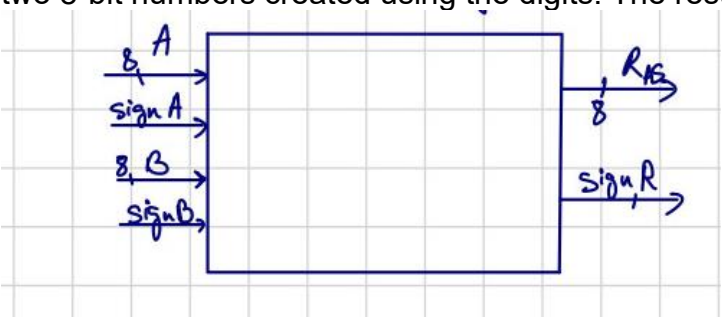
2.2 Control and Execution Unit



2.3 RESOURCES

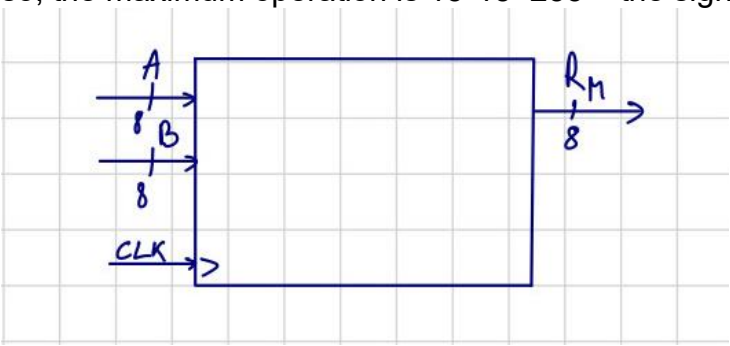
1) **Addition/Subtraction**

It represents the main component and is designed totally structural. This adds or subtracts two 8-bit numbers created using the digits. The result will be on 8 bits + the sign bit.



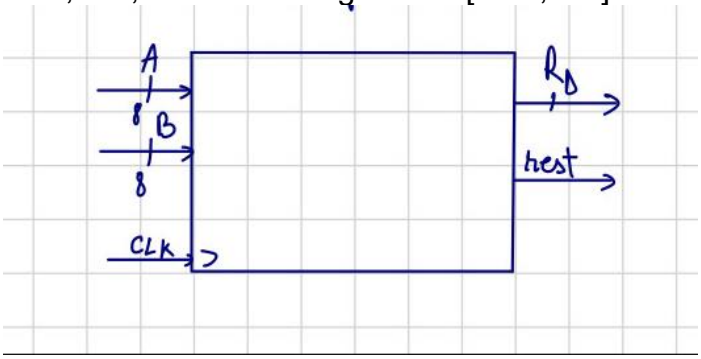
2) **Multiplication**

This is the main component that multiplies two 8-bit numbers. The result will be on 8 bits so, the maximum operation is 15*15=255 + the sign



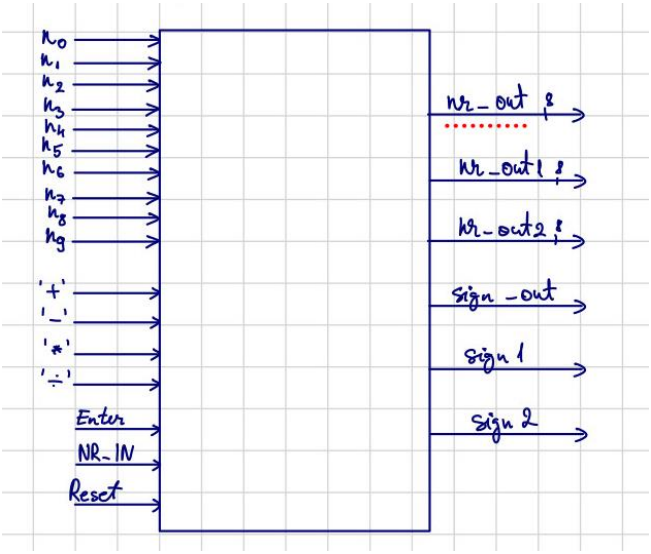
3) **Division**

This is the main component that divides two 8-bit numbers. The result will be on 8 bits, 4 bits for the remainder and one bit for the sign. There are some forbidden operations like : 0/0, X/0, where X- integer from [-255,255]



4) **Encoder**

This is the main component that converts the introduced number to binary. The maximum number of digits introduced is 3 for each number, then the next number is required (successive operations), because the selection interval is [-255,255]. The result is calculated and can be later modified if another digit is introduced.



5) Decision

This is the component that gives the final result by checking the ENTER switch.

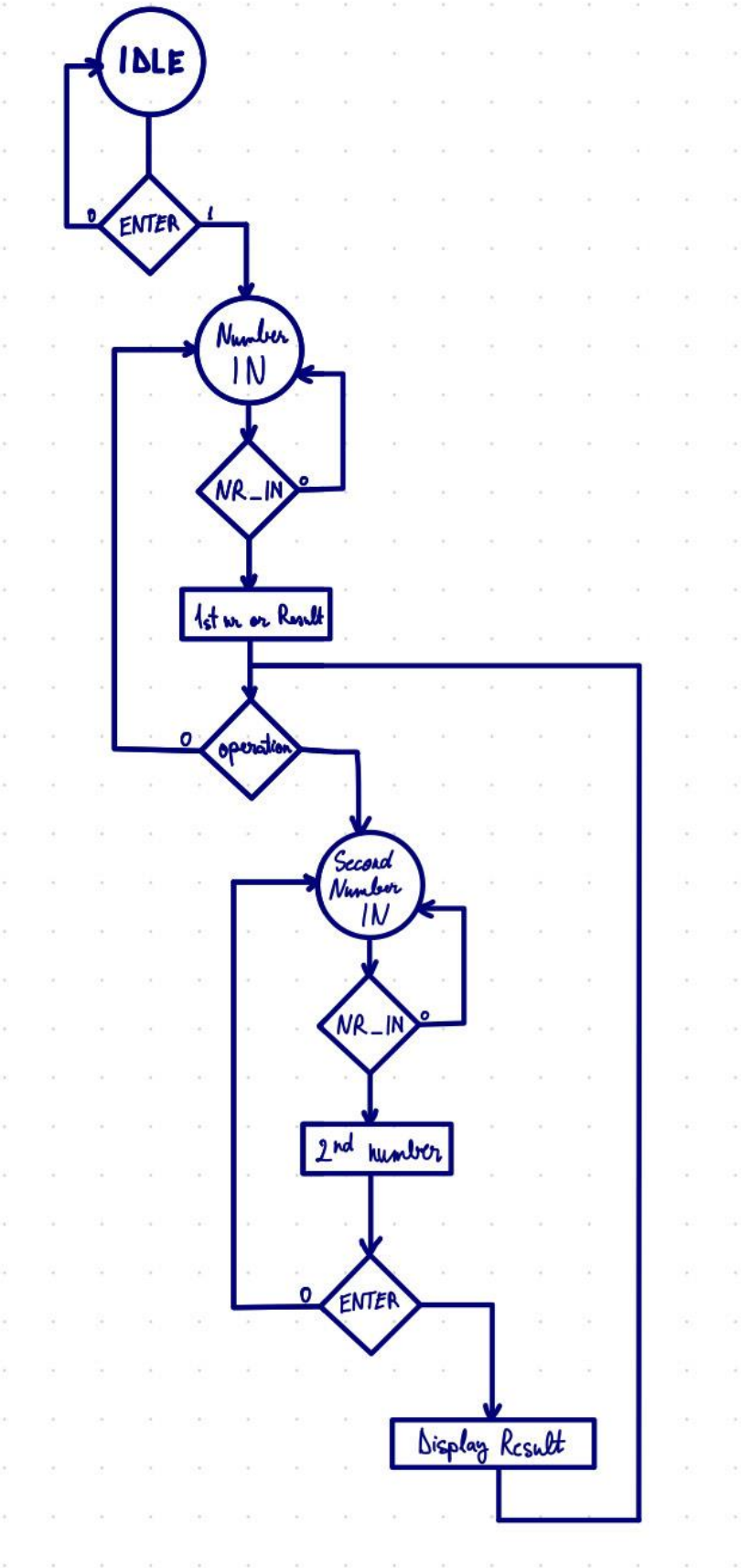


6) 7-segment

This is the component that decodes the final result and displays it on the 7-segment display of the board. The number is divided by 10 so that we obtain every decimal. According to their order and value, they are shown on the display



2.4 State diagram



The schematic illustrates the cyclic functionality of the circuit. The algorithm continuously repeats until a specific input is activated (for example: ENTER and NR_IN — leading to the content being modified or displayed).

The first condition to check is the **RESET** switch. If it is active, the numbers are reset to 0; if not, the previous values are retained.

The **NR_IN** input represents the confirmation of a digit, triggering the construction of a number with that digit. If there is no impulse on the **NR_IN** input, the computation proceeds with the existing values of **NR1** and **NR2**. The result of each computation is stored in a dedicated signal.

Next, the algorithm checks whether the **ENTER** input is active. If it is, it determines which operation is selected:

- If **'+** or **-'** is active: the final result takes the value from the Addition/Subtraction signal.
- If **'*'** is active: the final result takes the value from the Multiplication signal.
- If **'/'** is active: the final result takes the value from the Division signal.

This decision-making process occurs within the **Decision** component.

If the **ENTER** input is inactive ('0'), the system skips the computation step and directly proceeds to display the result on the 7-segment display and the additional output bits. If no new input is applied, the display maintains the previously shown value (initially 0).

Following this, the algorithm re-evaluates whether the **NR_IN** and **ENTER** inputs are active (indicated by the *NR_IN* decision on the right side of the schematic). After the result is displayed due to the **ENTER** input activation, the **NR_IN** input is also triggered. If both conditions are satisfied, the computation continues: **NR1** is updated with the result, and **NR2** is reset to 0. This mechanism supports successive operations.

The process restarts upon activation of the **RESET** switch.

This summarizes the general algorithm and steps followed in the development of this project.

3. USER MANUAL

First, we push the RESET button to initialize the board. Then, we choose a digit, followed by the activation of the NR_IN switch. After the number is constructed, we turn off the switches, select an operation and continue by introducing the second number. At the end, we turn on the ENTER switch and the result is displayed. After the result is displayed we can introduce another number in the same way as we did previously and after choosing another operation, the calculator will compute the result with the previous result and the newly introduced number.

4. DESIGN SOLUTIONS JUSTIFICATION

I chose this modular VHDL implementation because it was straightforward and effective for the Nexys A7-100T FPGA. I first developed the main components (codificator.vhd, sumator_scazator.vhd, Inmultire.vhd, Impartire.vhd, 7sgmnt.vhd, decision.vhd, etc.) and then connected them in the main component. Operations are computed continuously, with the result stable until the Enter switch is activated.

5. FURTHER DEVELOPMENT

1) **Merging Enter and NR_IN Inputs:**

The calculator uses separate Enter (to compute results) and NR_IN (to confirm digits) inputs, requiring two buttons. Combining them into one button would simplify the design. The calculator could detect if the button confirms a digit or triggers a calculation based on whether an operator (plus, minus, ori, div) is active. This reduces hardware complexity and makes the calculator easier to use.

2) **Using a Separate Keyboard for Input:**

Currently, digits (0–9) and operators are entered via switches. A separate keyboard (e.g. a USB keyboard interfaced with the FPGA) could be added to input digits and operators directly. This would make entering numbers and operations faster and more intuitive.

6. BIBLIOGRAPHY

- Octavian Augustin Creț : Digital Systems Design Course (2025), Technical University of Cluj-Napoca
- Octavian Augustin Creț : Logic Design Course (2024), Technical University of Cluj-Napoca
- Diana Irena Pop : Digital Systems Design Laboratory Works (2025), Technical University of Cluj-Napoca
- Diana Irena Pop : Logic Design Laboratory Works (2024), Technical University of Cluj-Napoca